Verzweigungen - switch

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Verzweigungen in Java
- Varianten von if-else

Was wird in diesem Foliensatz besprochen?

- switch-Anweisung
- switch-Ausdruck
- Unterschiede zwischen if-else und switch

Was lernen Sie kennen?

- switch-Anweisung in Java
- switch-Ausdruck in Java
- Auswahl zwischen switch und if-else

switch-Anweisung

- Allgemeine Form

```
switch (Ausdruck) {
    case Wert_1:
            Anweisung(en)_1
            break;
    case Wert_2:
            Anweisung(en)_2
            break;
    …
    case Wert_n:
            Anweisung(en)_n
            break;
    default:
            Anweisung(en)_default
            break;            /* Nicht notwendig, aber hilfreich */
}
```

switch – allgemeine Beschreibung

```
switch (Ausdruck) {
    case Wert_1:
        Anweisung(en)_1
        break;
    case Wert_2:
        Anweisung(en)_2
        break;
    …
    case Wert_n:
        Anweisung(en)_n
        break;
    default:
        Anweisung(en)_default
        break;
}
```

Ausdruck wird ausgewertet

Erlaubte Datentypen (von den bisher besprochenen): short, byte, int, char oder String

Auswahl einer Alternative

Wert des Ausdrucks entspricht dem **konstanten Wert** hinter einer case-Marke

switch – break

```
switch (Ausdruck) {
    case Wert_1:
        Anweisung(en)_1
        break;
    case Wert_2:
        Anweisung(en)_2
        break;
    …
    case Wert_n:
        Anweisung(en)_n
        break;
    default:
        Anweisung(en)_default
        break;
}
```

Sorgt dafür, dass **nicht** zur folgenden case-Marke weitergesprungen wird

- switch-Anweisung wird verlassen

Ohne break können zwei oder mehrere case-Marken verbunden werden

- Alle nachfolgenden Anweisungen werden bis zum nächsten break oder dem Ende der switch-Anweisung abgearbeitet
- „**Fall-through**“

switch – default

```
switch (Ausdruck) {
    case Wert_1:
        Anweisung(en)_1
        break;
    case Wert_2:
        Anweisung(en)_2
        break;
    …
    case Wert_n:
        Anweisung(en)_n
        break;
    default:
        Anweisung(en)_default
        break;
}
```

default-Marke

- ist optional
- wird ausgeführt, wenn keine case-Marke passt
- ohne default ist es möglich, dass keine Anweisungen ausgeführt werden

Beispiele (switch)

```java
public class GradingExample3 {

    public static void main(String[] args) {
        int grade = 2;  // Test data

        String text;
        switch (grade) {
            case 1:
                text = " excellent ";
                break;
            case 2:
                text = " good ";
                break;
            case 3:
                text = " satisfactory ";
                break;
            case 4:
                text = " sufficient ";
                break;
            case 5:
                text = " insufficient ";
                break;
            default:
                text = " no grade ";
        }

        System.out.println("Your grade: " + text);
    }
}
```

```
Your grade: good
```

Beispiel mit switch statt Mehrfachverzweigung
Variante von GradingExample1.java

Beispiele (switch)

```java
public class GradingExample4 {

    public static void main(String[] args) {
        int grade = 2;  // Test data
        String text;

        switch (grade) {
            case 1:
                text = " with distinction ";
                break;
            case 2:
            case 3:
            case 4:
                text = " passed ";
                break;
            case 5:
                text = " not passed ";
                break;
            default:
                text = " no grade ";
        }

        System.out.println("Your grade:" + text);
    }

}
```

```
Your grade: passed
```

Alternatives Beispiel mit switch - benutzt „Fall-through“

switch-Ausdruck (Java 14)

```java
public class GradingExample3 {

    public static void main(String[] args) {
        int grade = 2;  // Test data

        String text;
        switch (grade) {
            case 1:
                text = " excellent ";
                break;
            case 2:
                text = " good ";
                break;
            case 3:
                text = " satisfactory ";
                break;
            case 4:
                text = " sufficient ";
                break;
            case 5:
                text = " insufficient ";
                break;
            default:
                text = " no grade ";
        }

        System.out.println("Your grade: " + text);
    }

}
```

Seit Java 14

```java
public class SwitchExpressionTest {

    public static void main(String[] args) {
        int grade = 1;  // Test data

        String text = switch (grade) {
            case 1 -> " excellent ";
            case 2 -> " good ";
            case 3 -> " satisfactory ";
            case 4 -> " sufficient ";
            case 5 -> " insufficient ";
            default -> " no grade ";
        };

        System.out.println("Your grade:" + text);
    }

}
```

switch erzeugt einen Wert, der einer Variable zugewiesen wird

switch-Ausdruck

Syntaktische Änderung („->“ statt „:“)

Mehrere Werte hinter case-Klausel möglich

Kein break mehr

- Es existiert kein „Fall-through“!

switch gibt einen Wert zurück

- Hilfsvariablen können vermieden werden

Compiler überprüft auf Vollständigkeit

- Alle möglichen Werte müssen abgedeckt werden (direkt oder über default)

Schlüsselwort yield bei switch (1)

```
…
int grade = 3;  // Test data
String text = switch (grade) {
    case 1:
        yield " with distinction ";
    case 2, 3, 4:
        yield " passed ";
    case 5:
        yield " not passed ";
    default:
        yield " no grade ";
};
…
```

Alternative Schreibweise (ähnlich zu alter Syntax)

Schlüsselwort yield bei switch (2)

```
…
int grade = 1;  // Test data
String text = switch (grade) {
    case 1 -> {
        String message = " with distinction ";
        message = "***" + message + "***";
        yield message;
    }
    case 2, 3, 4 -> " passed ";
    case 5 -> " not passed ";
    default -> " no grade ";
};
…
```

Rückgabe eines Wertes bei mehreren Anweisungen in einem Block

Vergleich von if und switch

if	switch

• Einfache Auswahlsituationen
  • Anweisung(en) ausführen oder nicht bzw. alternative Anweisungen ausführen
• Komplexere Ausdrücke in Bedingungen möglich
  • Z.B. if (counter > 10 && counter < 30)…
	• Anhand mehrerer konstanter Werte verzweigen